A tour of PyNet2Plan

Lab handout 1

October 2026

The numbers in the figures refer to the numbered paragraphs below each figure.

PyNet2Plan is a small network planning tool written in Python for this course. It is inspired by Net2Plan, but all of its parts are ordinary Python code that can be read and modified. A network is represented as a NetworkX directed graph (`networkx.DiGraph`) in which every node is a city and every edge is a unidirectional optical link, characterised by a number of fibers (one by default), a capacity (number of wavelengths λ of each fiber) and a length in kilometres. The traffic is a list of demands, each with an origin, a destination and a number of wavelengths; it is read from a CSV file or generated at random from a seed. Each algorithm is a Python file in the folder `formulations/`: ILP formulations are written with PuLP, heuristics with NetworkX. The result of every run is saved as a folder of CSV files in `results/`, which the *Results* tab opens again.

This handout follows a typical session: a network and its traffic are opened, an algorithm is run, the results are inspected and several runs are compared. It is useful to repeat the same steps on your own computer while reading.

The figures were taken with *View* > *Text size* > *Large*. On macOS and Windows the window looks slightly different, and the solver may choose other routes (section 13).

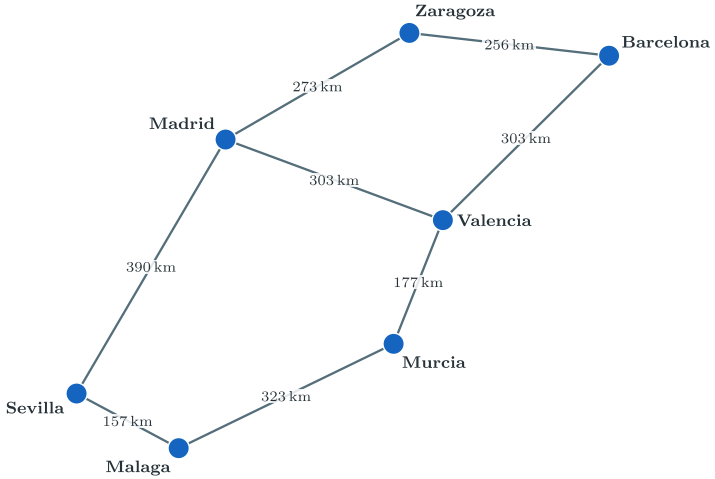
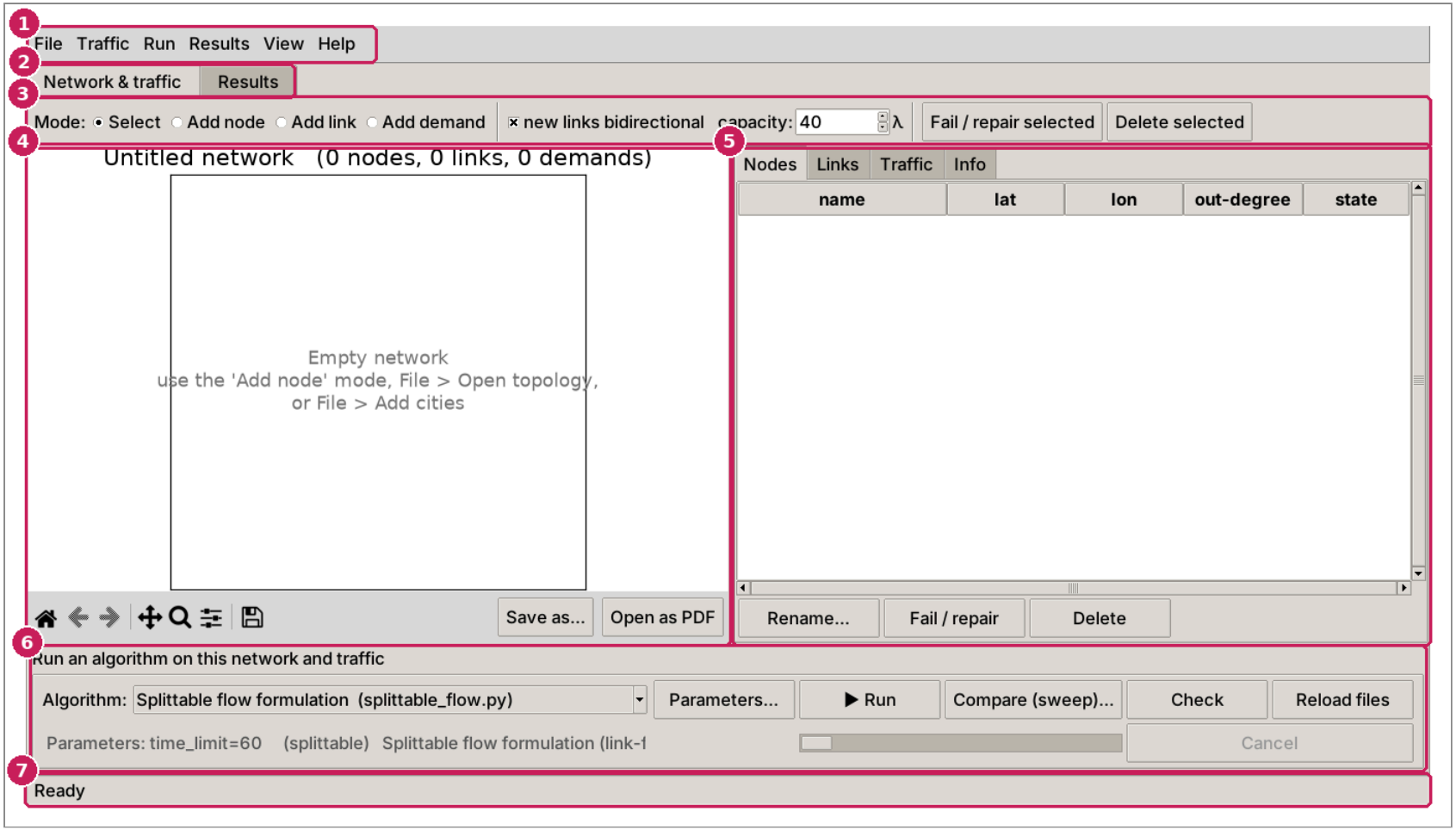


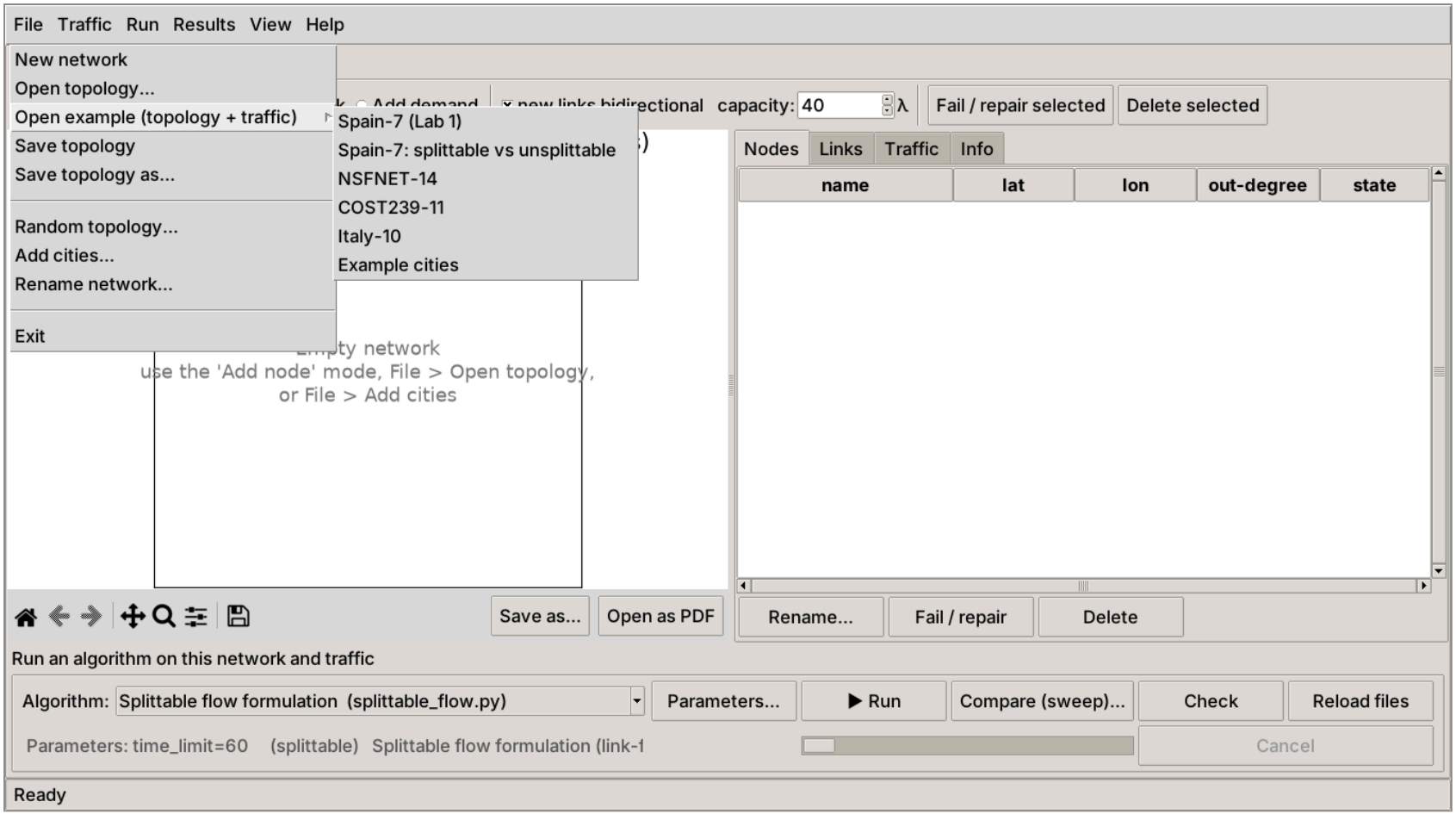
Figure 1. The Spain-7 network used in this handout: 7 nodes and 8 pairs of opposite links (16 unidirectional links).

1. The main window



1. The menus. *File* contains the network commands, *Traffic* the traffic matrices, *Run* the algorithms and the solver options, *Results* the saved runs, *View* the display options and the text size, *Help* a short guide and the installation check.
2. The two tabs: *Network & traffic* to prepare the inputs, *Results* to inspect the saved runs.
3. The editing modes. With *Add node*, *Add link* and *Add demand* a network can be drawn directly on the map; *Fail / repair selected* switches a node or a link off and on.
4. The map of the network. The toolbar below it and the mouse wheel zoom and pan the map; *Save as...* saves the map, as it appears, as a PDF, PNG or SVG file, and *Open as PDF* shows it in a PDF file (in a temporary folder).
5. The tables of nodes, links and traffic, and a summary of the network (*Info*).
6. The run bar, used to choose an algorithm, set its parameters, run it, compare several runs and check an exercise.
7. The status bar, which reports the result of the last action.

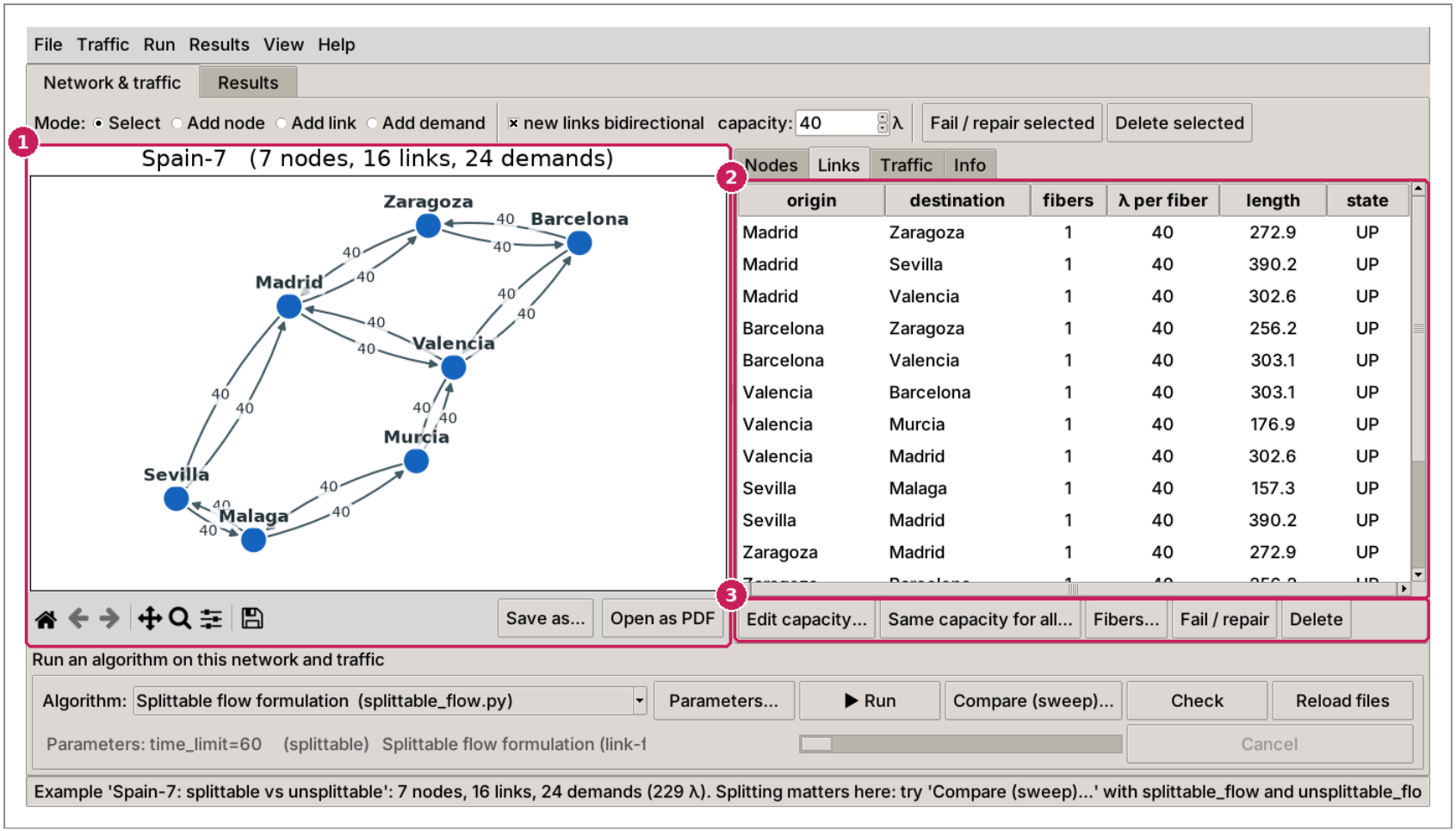
2. Opening a network and its traffic



The menu *File > Open example* opens one of the reference networks in the folder `data/` together with its traffic matrix. For this handout choose *Spain-7: splittable vs unsplittable*.

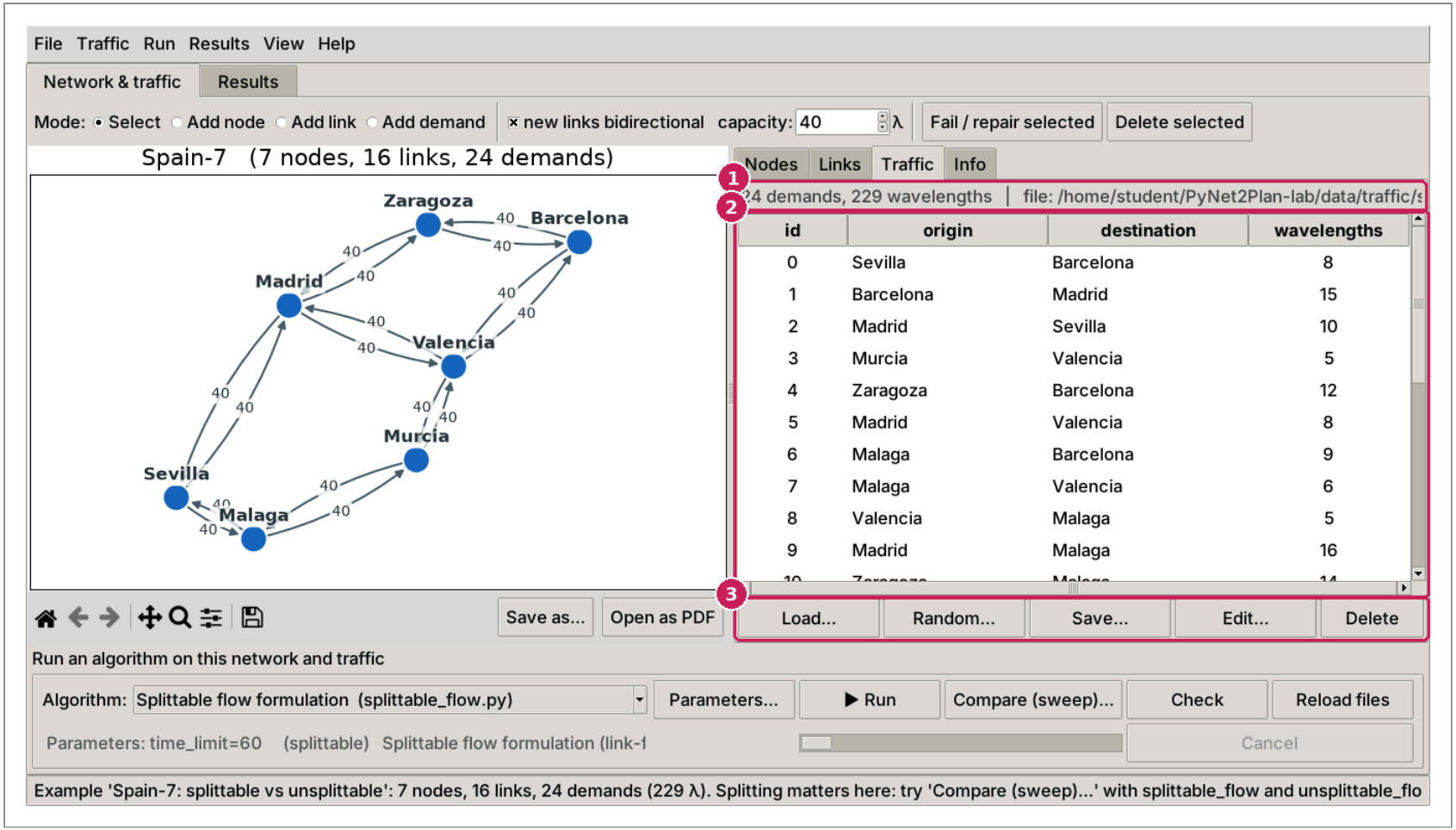
Other networks are opened with *File > Open topology...* (JSON, GraphML or Net2Plan `.n2p` files in `data/topologies/`) and other traffic matrices with *Traffic > Open traffic matrix...* (CSV files in `data/traffic/`). A network can also be drawn on the map, built from the names of real cities (*File > Add cities...*) or generated at random (*File > Random topology...*).

3. Nodes and links



1. Each arrow on the map is a unidirectional link, labelled with its capacity (here 40 λ; a link with two fibers of 40 λ is labelled 2×40 and drawn thicker). The two directions between two cities are two different links and are drawn as two arcs.
2. The *Links* table lists origin, destination, number of fibers, wavelengths per fiber, length and state of every link. A click on a column title sorts the table; a click on a row selects the link on the map.
3. *Edit capacity...* changes the wavelengths per fiber of the selected links and *Same capacity for all...* assigns the same value C to every link, which is often needed in the experiments. *Fibers...* sets the number of fibers; all the networks of the lab have one fiber per link.

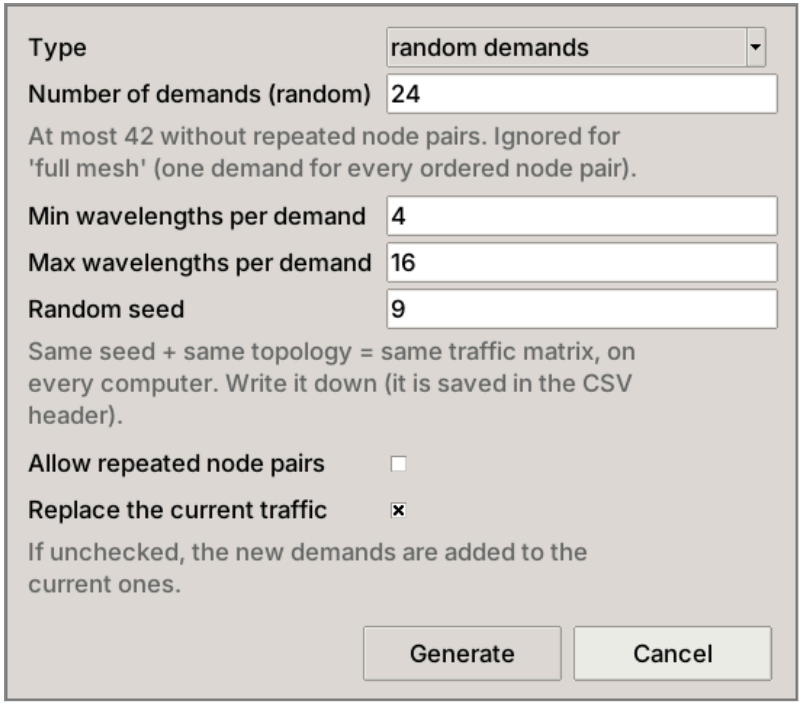
4. The traffic matrix



1. The summary line gives the number of demands (24), the total traffic (229 wavelengths) and the file the demands were read from.
2. Each row of the table is a demand, with its identifier, origin, destination and number of wavelengths (between 4 and 16 in this matrix).
3. The buttons load, generate, save, edit and delete demands. A saved traffic matrix is an ordinary CSV file that can also be opened with a spreadsheet.

A traffic file is either a list of demands, with the columns `id,origin,destination,traffic`, or an N × N matrix with one row and one column per node. The program recognises the format automatically.

5. Random traffic and the seed



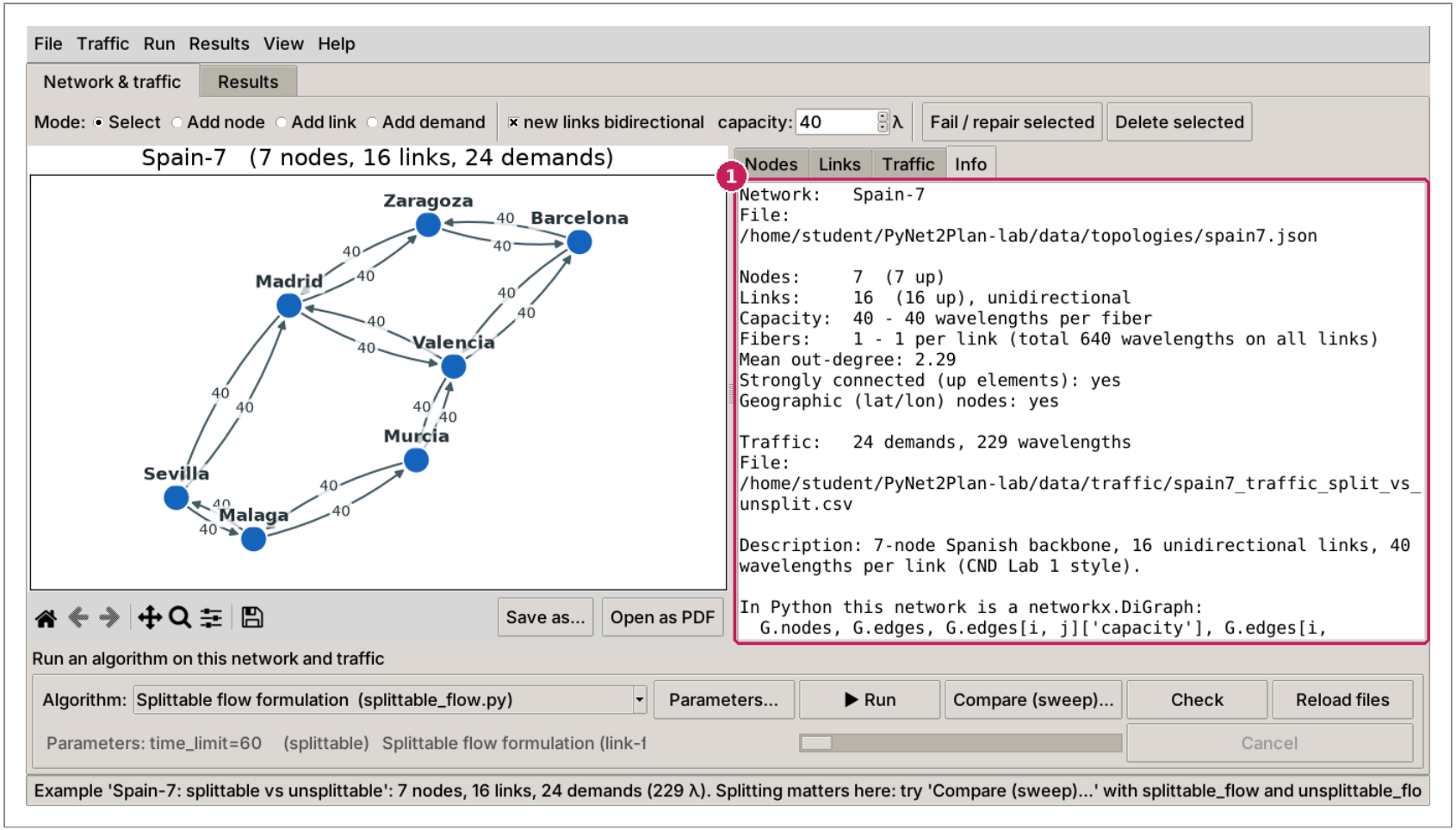
The dialog *Traffic > Generate random traffic...*

Random traffic is generated from a seed. The same seed applied to the same network produces exactly the same demands on every computer. The values shown in the figure (24 demands of 4 to 16 wavelengths, seed 9) reproduce the traffic matrix used in this handout.

The type is either *random demands*, with origin and destination drawn at random, or *full mesh*, with one demand for every ordered pair of nodes. The number of demands and the minimum and maximum number of wavelengths define the size of the matrix.

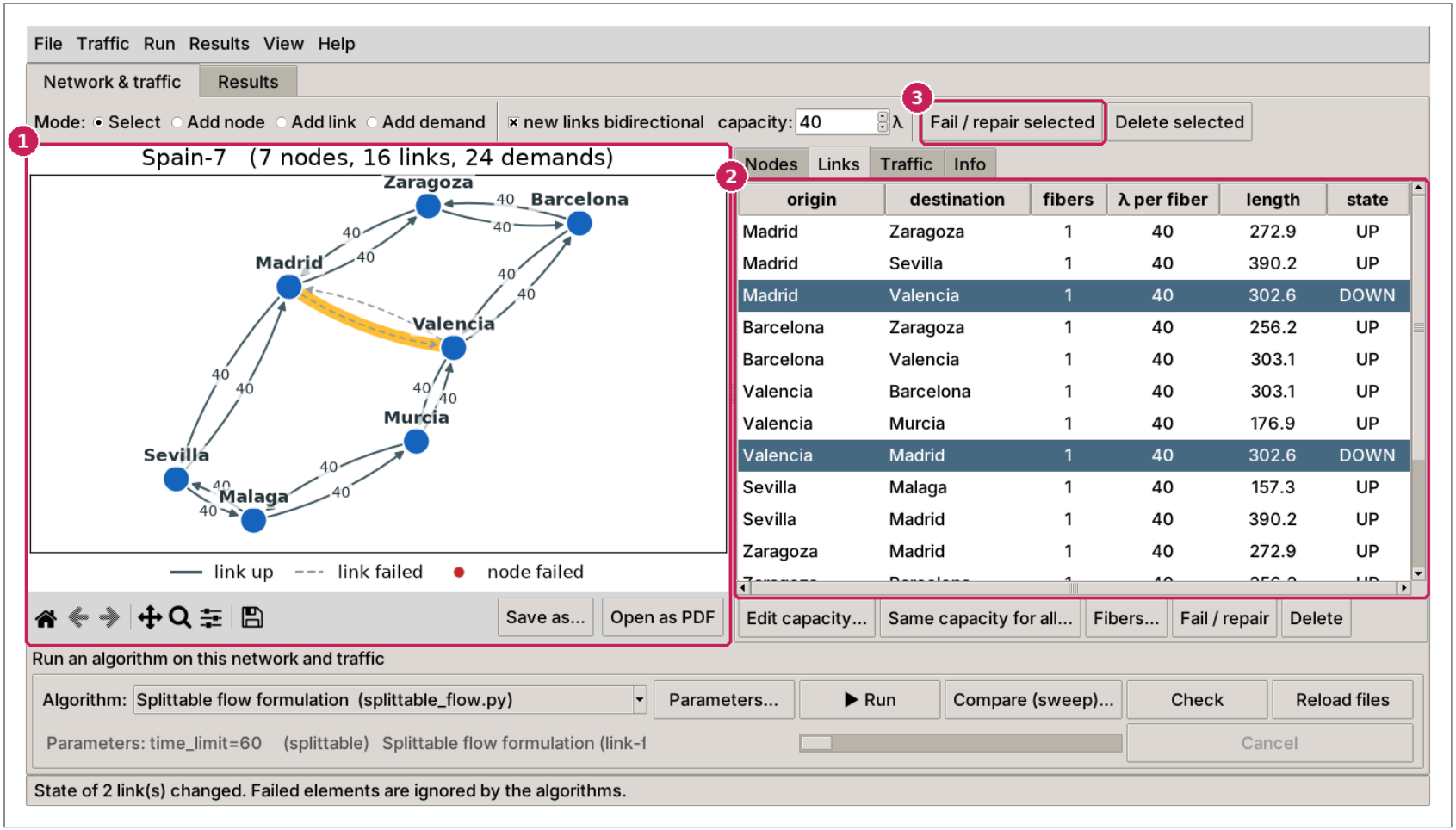
An experiment can be repeated only if its inputs are known. For this reason the seed should be written down, or the traffic saved with *Traffic > Save traffic as...*; the seed is also recorded in the first lines of the saved CSV file. PyNet2Plan stores a copy of the network, the traffic and the algorithm with every result.

6. The Info tab



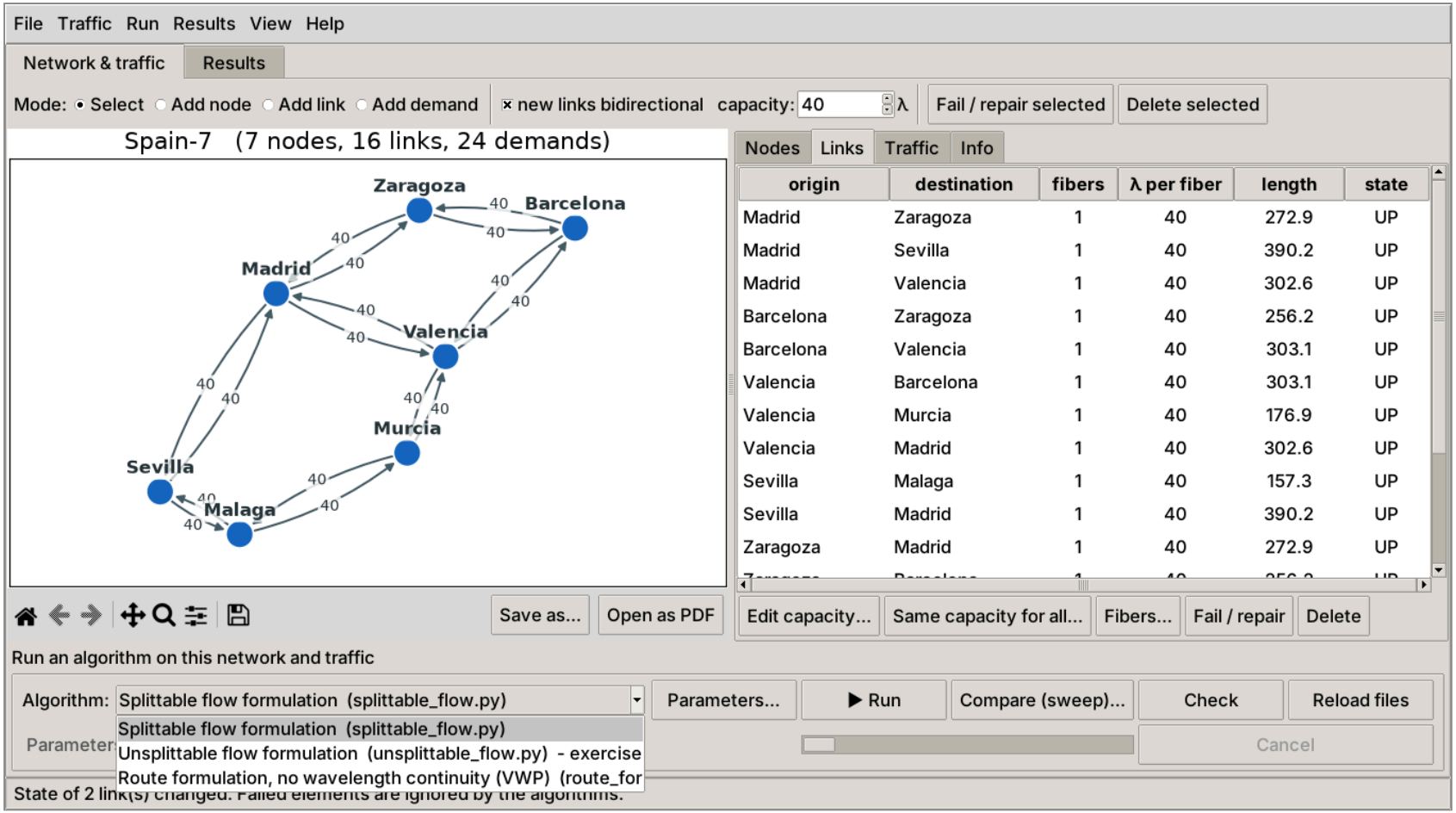
1 The *Info* tab summarises the network (nodes, links, capacities, degrees and connectivity) and recalls how the same data are accessed from Python, for example with `G.nodes`, `G.edges`, `G.edges[i, j]["capacity"]` or `G.out_edges(n)`. These NetworkX functions are the subject of handout 2.

7. Failures



1. A failed link is drawn as a dashed grey line. Failed nodes and links are removed from the network that is passed to the algorithms.
2. A fibre cut interrupts both directions of a link, so both rows must be selected in the table (Ctrl+click, or Cmd+click on macOS). Their state becomes DOWN.
3. *Fail / repair selected* switches the selected nodes or links off, and on again.

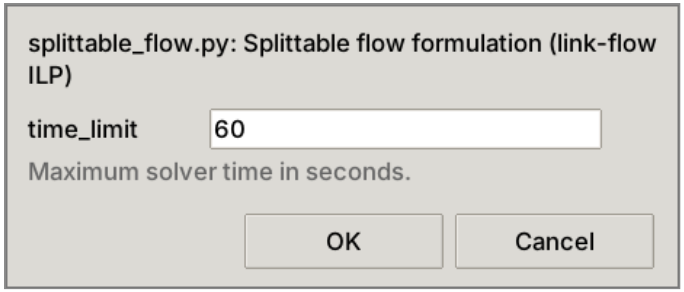
8. Choosing the algorithm



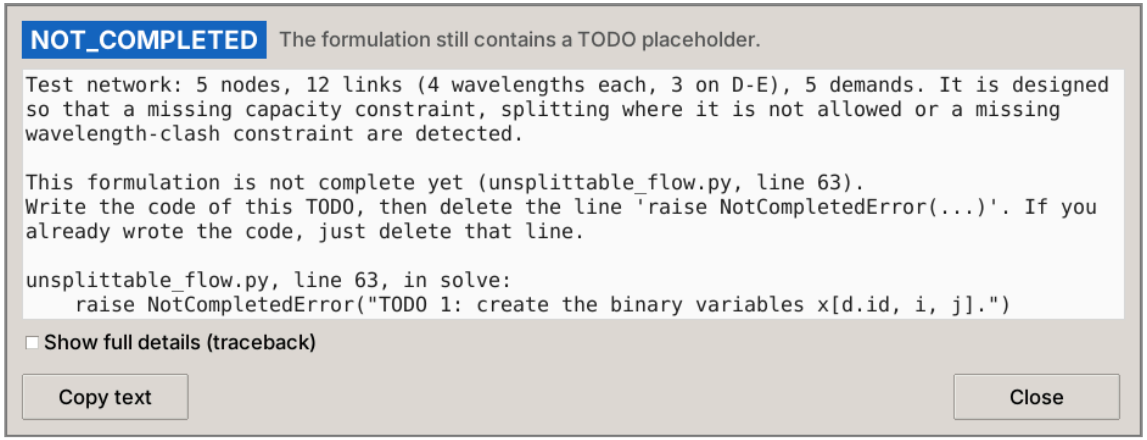
The list of algorithms contains one entry for each Python file in the folder `formulations/`.

One algorithm is complete: the splittable flow formulation, used in this lab. In the figure the list also contains the unsplittable flow formulation and the route formulation, marked *exercise, N TODO left*: they are completed in Lab 2 and are added to your copy before that lab with `git pull` (handout 0). The file of an algorithm is read again at every run, so the program does not need to be restarted after a formulation has been modified.

9. Parameters and the Check button



The parameters of the selected algorithm (*Parameters...*).

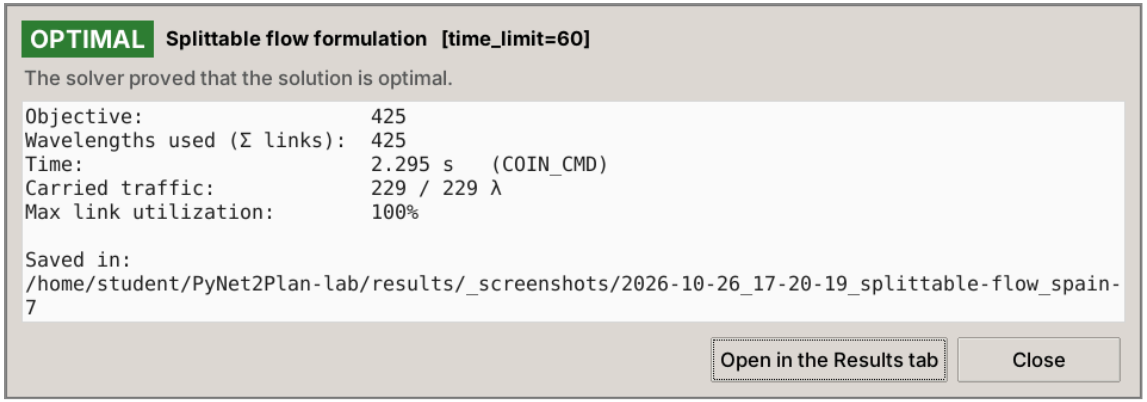


The answer of *Check* for an exercise that has not been completed.

The splittable flow formulation has a single parameter, `time_limit`, the maximum solver time in seconds. Its variables are always integer, since they count wavelengths. Other algorithms have further parameters, for example the number k of candidate paths of the route formulations.

The *Check* button runs the selected file on a small test network designed to reveal the most common mistakes, such as a missing capacity constraint, a demand split although splitting is not allowed, or two lightpaths on the same wavelength of a link. For an exercise that still contains its placeholders the answer is NOT_COMPLETED, together with the file name, the line number and the text of the first TODO.

10. Running the algorithm



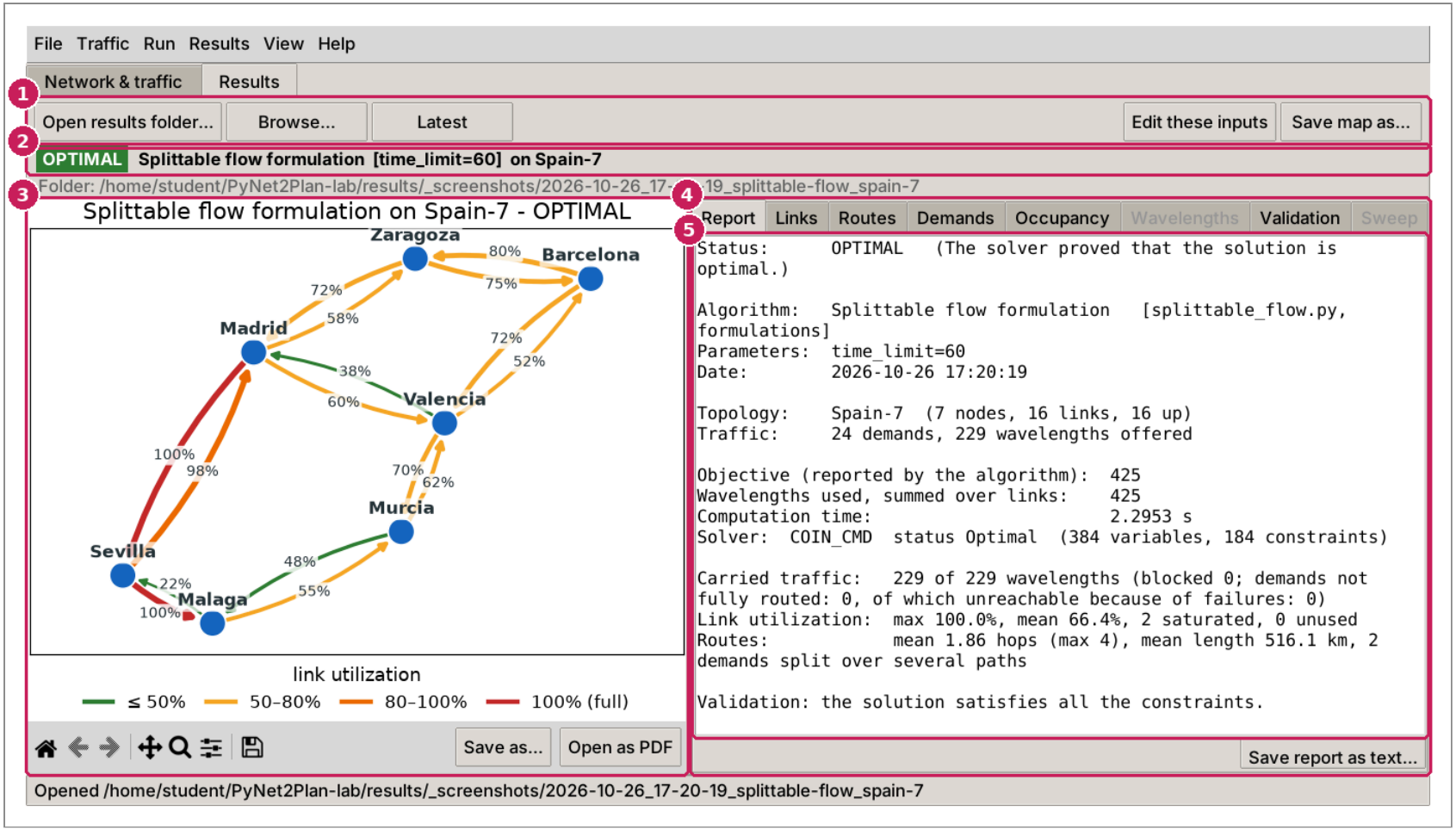
The summary shown after *Run* (or F5).

Run solves the problem in the background; meanwhile the progress bar moves and *Cancel* interrupts the computation. At the end a summary is displayed. Its first line is the status: OPTIMAL means that the solver has proved that no better solution exists. The objective, here 425, is the total number of wavelengths used, summed over all links. The summary also reports the computation time, the solver (CBC appears as COIN_CMD in PuLP), the carried traffic, the utilization of the most loaded link and the folder where the run was saved.

Before being saved, every solution is checked independently of the algorithm: flow conservation, link capacities and the rules on splitting are verified again. The button *Open in the Results tab* displays the run. The possible statuses are the following.

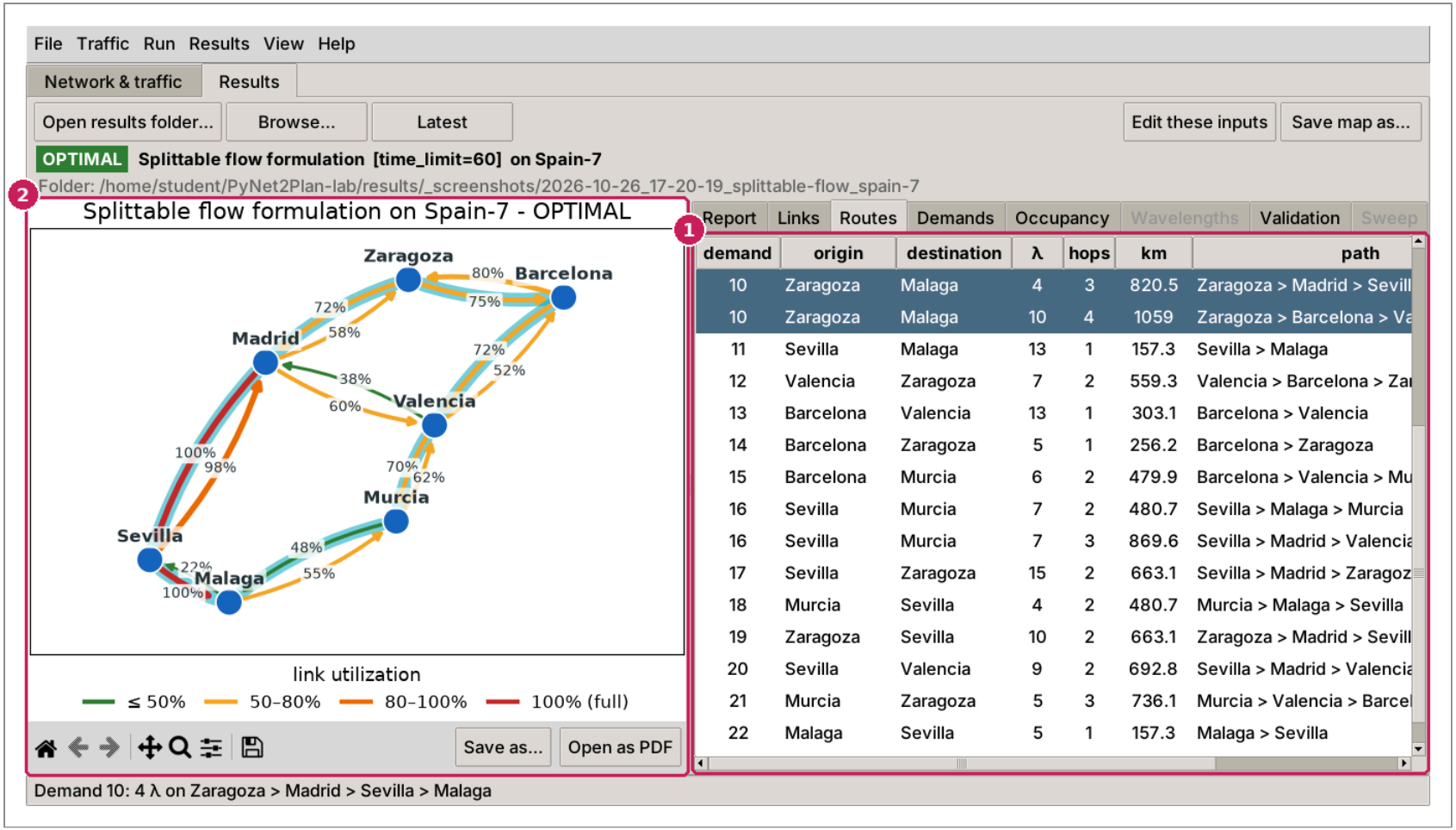
status	**meaning**
OPTIMAL, FEASIBLE	a valid solution, proved optimal or not (for example the solution of a heuristic)
INFEASIBLE	no routing satisfies all the constraints, for instance because the capacity is too small
INVALID	the model was solved, but its solution violates a constraint, so the formulation contains an error
NOT_COMPLETED, ERROR	a TODO is still present, or the code raised an error; file and line are reported

11. The Results tab



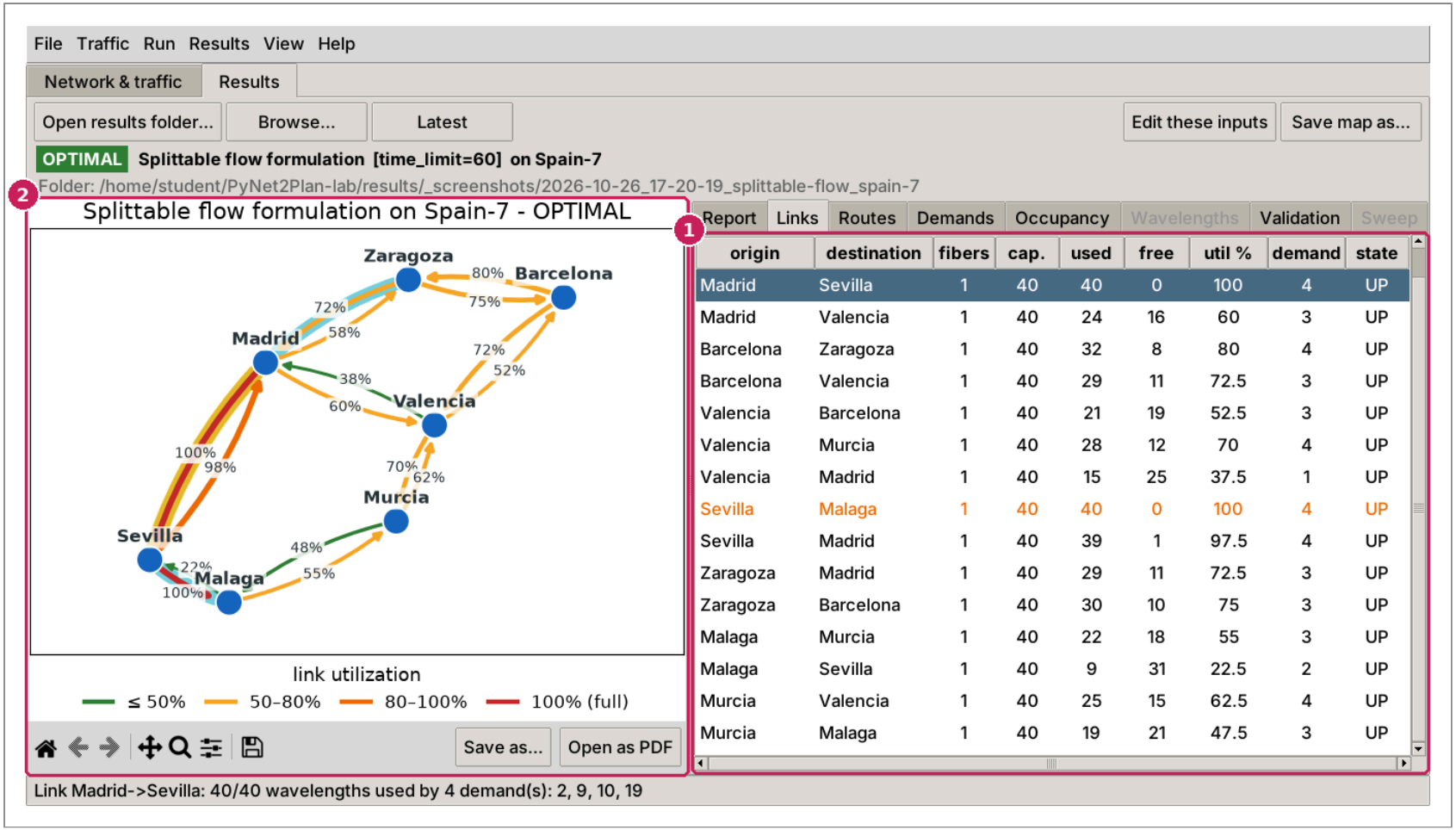
1. *Open results folder...*, *Browse...* (all the saved runs) and *Latest* select the run to display. *Edit these inputs* copies its network and traffic into the first tab.
2. Status, algorithm, parameters and network of the displayed run.
3. The map is coloured according to link utilization, defined as wavelengths used divided by capacity: green up to 50 %, yellow from 50 to 80 %, orange from 80 to 100 %, red when the link is full. The labels give the utilization of each link.
4. The tabs *Report*, *Links*, *Routes*, *Demands*, *Occupancy*, *Wavelengths* (only for formulations with wavelength continuity), *Validation* and *Sweep*.
5. The report describes the run: objective, time, solver, size of the model (384 variables and 184 constraints), utilization and routes.

12. Routes



1. The *Routes* table has one row for each path used by a demand, with its wavelengths, hops, length and sequence of nodes. A demand that appears in two rows has been split. In this run demand 10 (Zaragoza to Malaga) uses Zaragoza > Madrid > Sevilla > Malaga (4 λ) and Zaragoza > Barcelona > Valencia > Murcia > Malaga (10 λ).
2. The selected rows are highlighted on the map in light blue. Conversely, a click on a link of the map lists in the status bar the demands that use it.

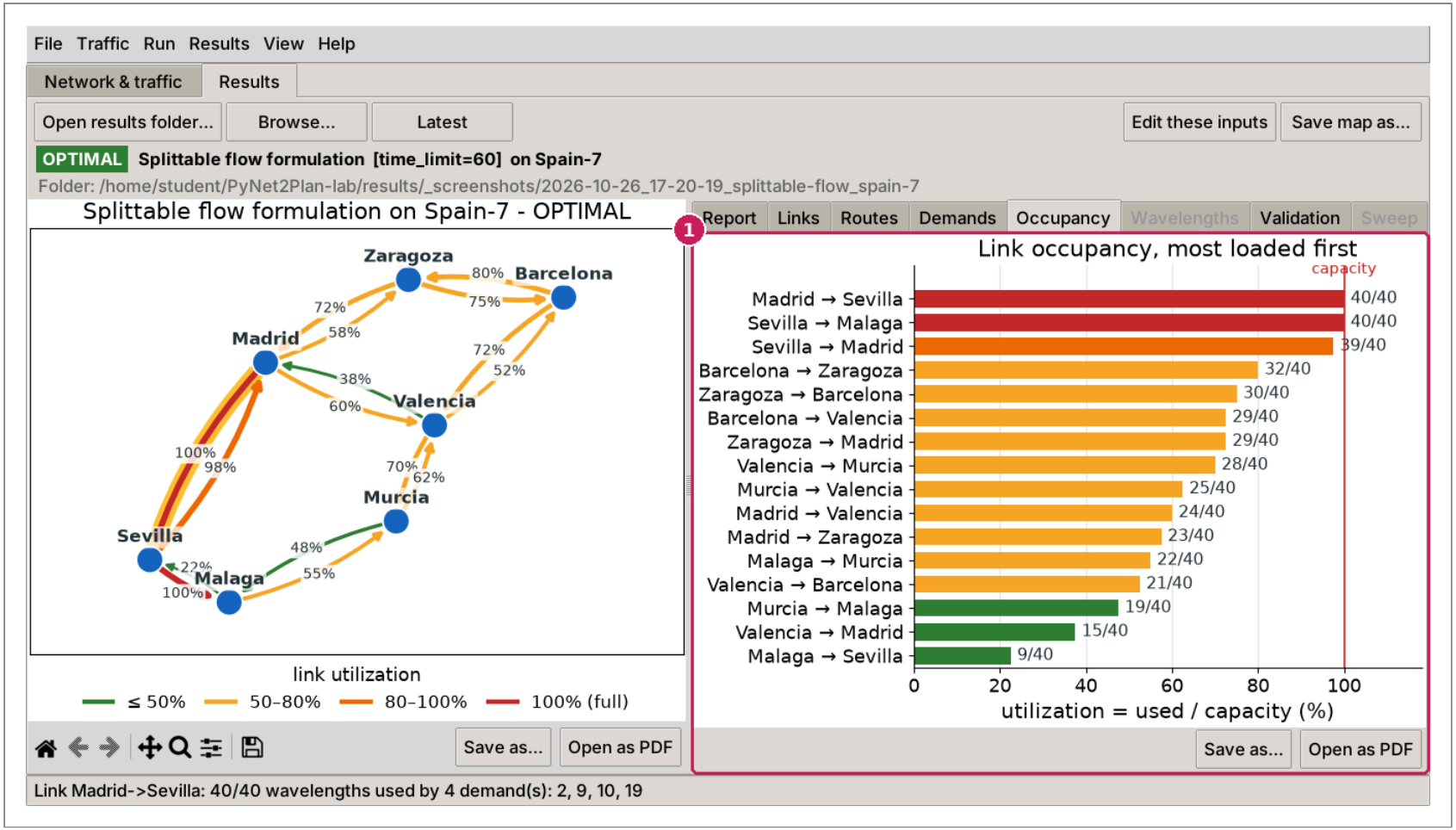
13. Links



1. The *Links* table gives the number of fibers, the capacity (fibers × wavelengths per fiber), the wavelengths used, the free wavelengths, the utilization and the number of demands of every link. Sorting by utilization shows the full links (here Madrid → Sevilla is one of them).
2. A selected link is highlighted on the map and its demands are listed in the status bar.

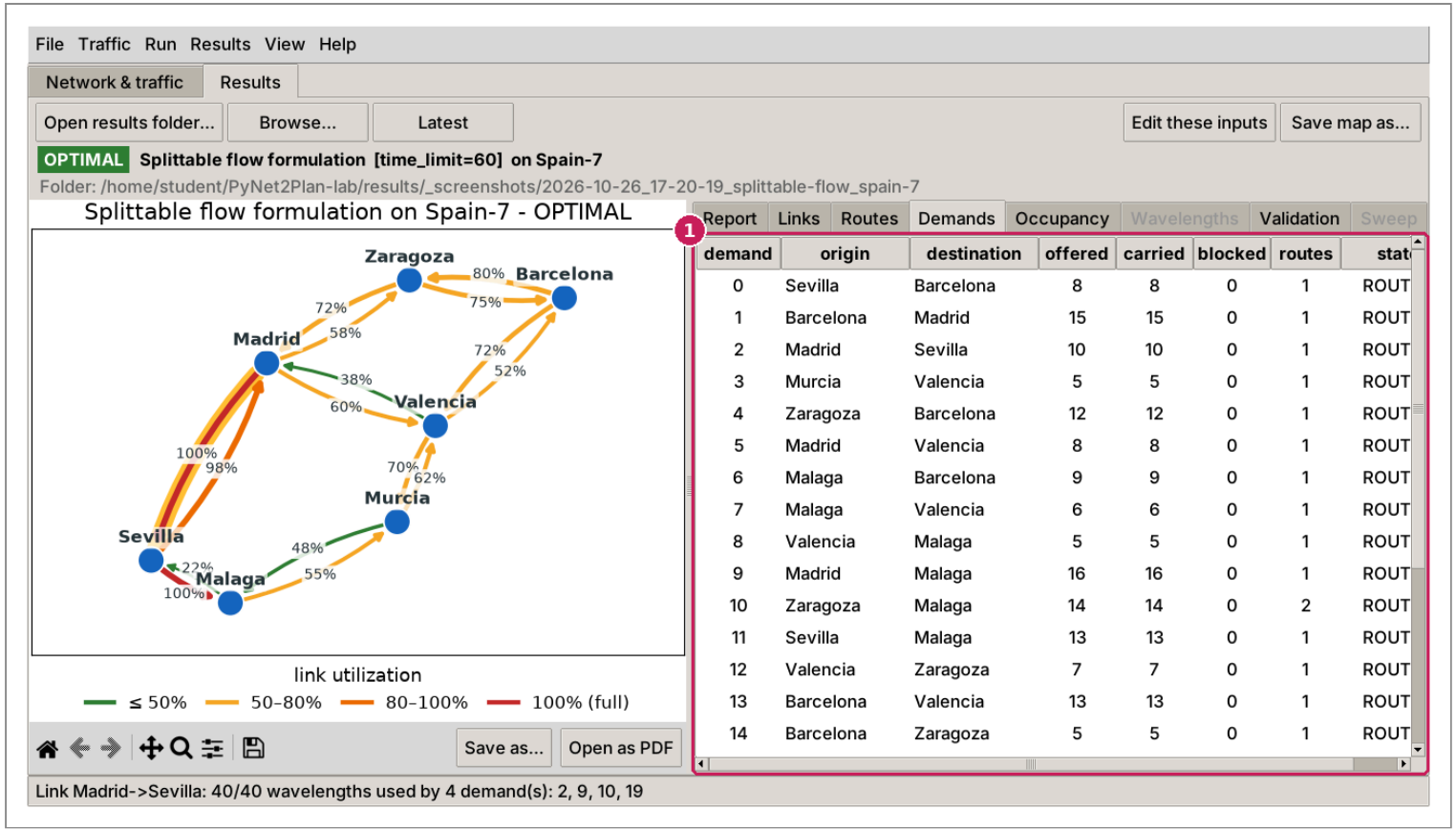
The optimal value of the objective (425 wavelengths) is unique, but in general several routings attain it. A different solver, or another version of the same solver, may therefore return a different optimal solution, with other split demands and other full links. The objective value is always the same.

14. The occupancy chart



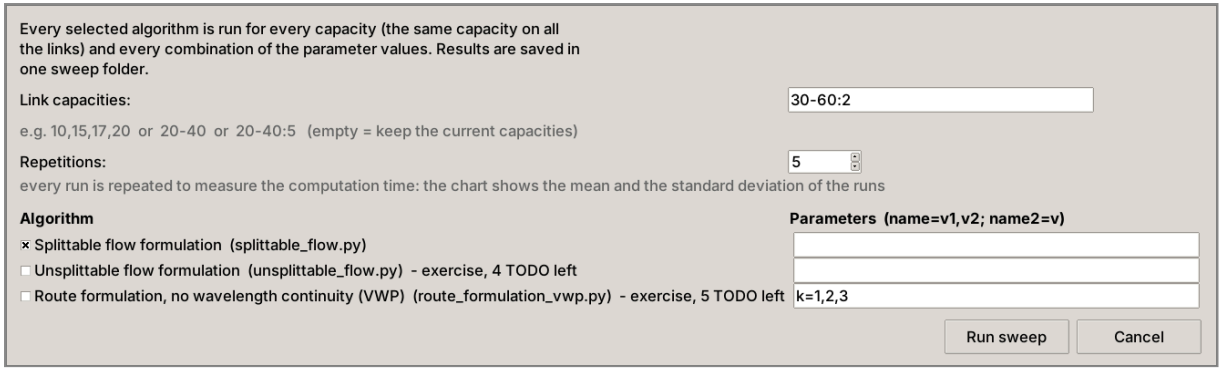
1 The *Occupancy* tab shows the links sorted from the most to the least loaded. Each bar is labelled with the wavelengths used and the capacity; the vertical line marks 100 %. The colours are the same as on the map. Below the chart, *Save as...* saves it as a PDF, PNG or SVG file, and *Open as PDF* saves it as `occupancy.pdf` in the folder of the run and opens it; the map and the *Wavelengths* chart have the same buttons.

15. Demands and validation



1 The *Demands* table gives, for every demand, the offered and carried wavelengths, the blocked wavelengths (a heuristic may block part of the traffic), the number of routes and the state. The *Validation* tab lists the problems found by the independent check of the solution; in this case there are none.

16. Comparing runs: the capacity sweep



The dialog *Compare (sweep)...* of the run bar.

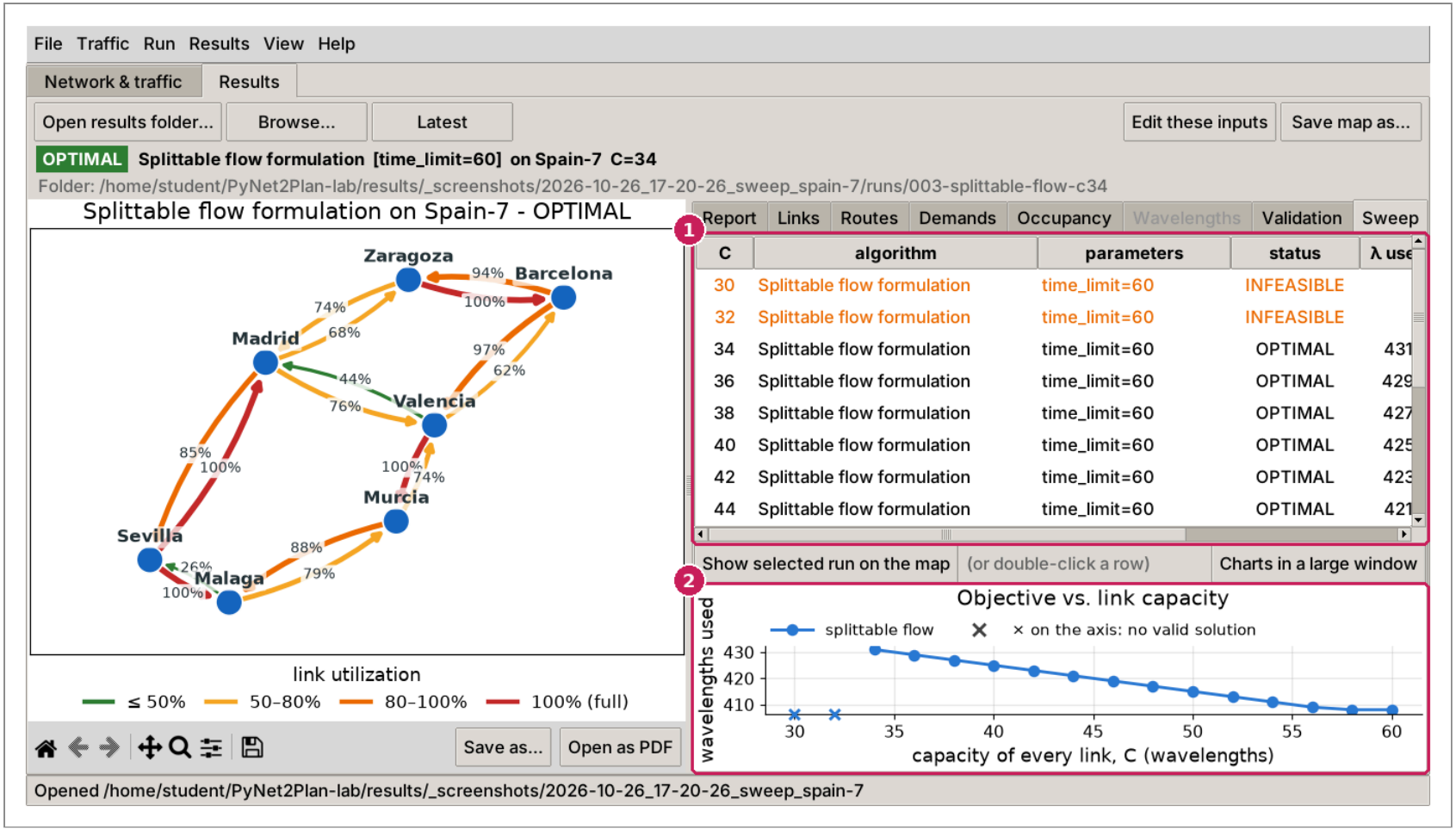
A sweep runs the selected algorithms for several values of the link capacity, the same value C being assigned to all links, and optionally for several values of their parameters.

The capacities are written as a list (`10,15,20`) or as a range: `30-60:2` stands for 30, 32, ..., 60. An empty field keeps the current capacities. In the parameter column a list of values, for example `k=1,2,3`, produces one series of runs for each value.

The computation time of the same run changes slightly every time it is repeated, because of the other programs running on the computer. *Repetitions* (5 by default) is the number of times each run is solved: the sweep reports the mean time and its standard deviation, which indicates how large these random variations are.

All the runs of a sweep are saved in a single folder, which contains the table `sweep_summary.csv` (one row per run) and the folders of the individual runs.

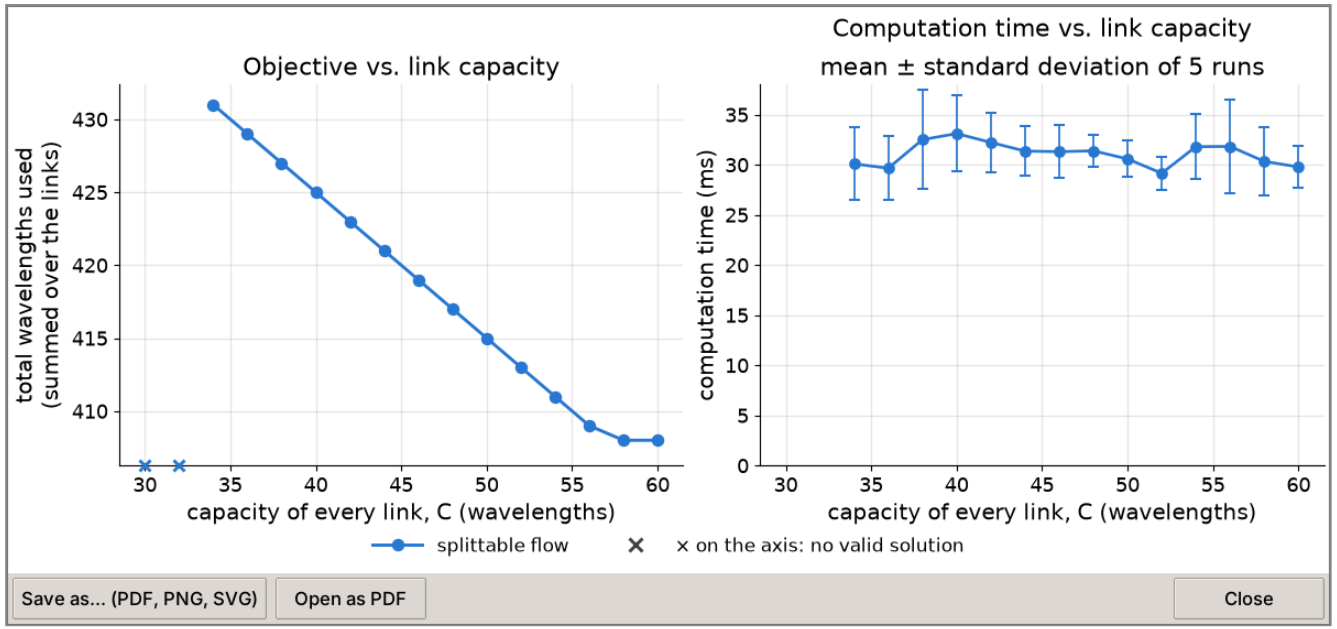
17. Results of a sweep



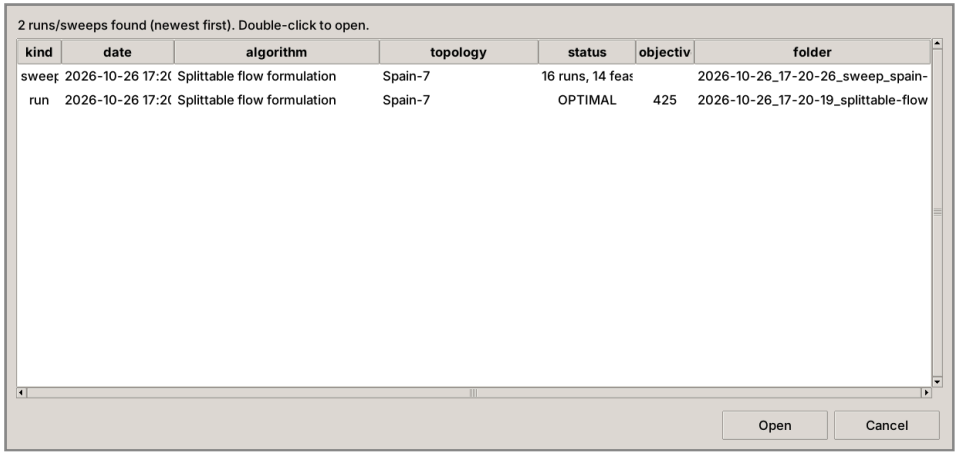
1. The table has one row per run, with capacity, algorithm, parameters, status, objective, mean time, its standard deviation (± ms) and number of repetitions; infeasible runs are shown in orange. A double click on a row displays that run on the map.

2. The chart shows the objective as a function of the capacity. Crosses at the bottom of the chart mark the capacities without a solution. *Charts in a large window* enlarges the chart, adds the computation times, with one legend below the two charts, and allows saving them as a PDF, PNG or SVG file or opening them as a PDF file.

18. Sweep charts and saved results



The window *Charts in a large window*: objective and computation time for each capacity. The points of the time chart are the mean of the repetitions, the error bars one standard deviation above and below.



The dialog *Results > Browse results...* lists all the runs and sweeps saved in `results/`, the most recent first.

19. The saved files

Each run is saved in its own folder with the following files.

file	content
`summary.csv`	algorithm, parameters, status, objective, time, solver, carried and blocked traffic, utilization (one row)
`links.csv`	fibers, wavelengths per fiber, capacity, used and free wavelengths, utilization and number of demands of each link
`routes.csv`	demand, path and wavelengths of each route, with the wavelength indices when they are assigned
`demands.csv`	offered, carried and blocked wavelengths and state of each demand
`wavelengths.csv`	the demand using each wavelength of each link (formulations with wavelength continuity)
`validation.csv`, `report.txt`	the result of the independent check and the text of the report
`topology.json`, `traffic.csv`, `formulation.py`	copies of the inputs and of the algorithm, so that the run can be repeated

A sweep folder contains `sweep_summary.csv` and the folders of its runs in `runs/`. The CSV files can be opened with Excel, LibreOffice or pandas, for example with `pd.read_csv("results/<folder>/links.csv")`, to prepare tables and charts for the lab reports. The buttons *Open as PDF* write the map and the charts of a run (`map.pdf`, `occupancy.pdf`, `wavelengths.pdf`) or of a sweep (`sweep.pdf`) in its folder, and the buttons *Save as...* save them under the name and in the folder chosen, as PDF, PNG or SVG files. PDF and SVG files are vector files, which remain sharp at any size and can be inserted directly in a report.

The size of all texts can be increased with *View* > *Text size*, which is convenient on a projector. *Help* > *Quick guide* summarises the main commands and *Help* > *Check installation* repeats the test of handout 0. All the operations of the graphical interface are also available from the command line, for example

```
python -m pynet2plan run -t data/topologies/spain7.json -d data/traffic/spain7_traffic_split_vs_unsplit.csv -a splittable_flow
```

(see `python -m pynet2plan --help`).

20. Exercises with the interface

The following exercises use only the graphical interface. Record the values you obtain: they are discussed at the end of the lab.

1. Open *File > Open example > NSFNET-14*. How many nodes, links and demands does the network have, and how many wavelengths does each link carry? Run the splittable flow formulation: what is the objective, and which link is the most loaded?
2. Fail the most loaded link of exercise 1 in both directions (*Links* tab, *Fail / repair selected*) and run the formulation again. How much does the objective increase, and which links are now the most loaded? Repair the link afterwards.
3. With *Compare (sweep)...*, find the smallest capacity, the same on all links, for which the splittable flow formulation is feasible on NSFNET-14.
4. Generate a random traffic matrix for NSFNET-14 with 30 demands of 4 to 16 wavelengths and seed 7 (*Traffic > Generate random traffic...*), save it, open it again and check that the summary line of the *Traffic* tab shows the same values. Run the splittable flow formulation with this matrix: is the result the same as that of your neighbour, who used the same seed?